

BFS Region-Growing: A Minimal Contiguous Bisection Baseline for BISECT

T.12 — Algorithm Design / Geometric Structure

Gio Della-Libera

Apportionment Research Group

`giodl@microsoft.com`

May 2026

Abstract

BFSGROWTH is BISECT’s smallest implemented structure-layer alternative to METIS-style bisection. At each bisection-tree node it selects one population-weighted seed, chooses a second seed at maximum BFS distance from the first, grows two frontier regions with a priority queue keyed by current district population, assigns any disconnected leftovers to the smaller side, and applies the same 200-step local boundary rebalance family used elsewhere in the platform. The only required inputs are adjacency, population, balance tolerance, base seed, and node path.

This paper corrects the older presentation of BFS Growth. The implementation is a recursive two-way bisection primitive, not a general k -seed simultaneous constructor. It is a baseline and warm-start candidate, not a certified compactness floor. Its strongest property is legibility: every accepted tract is reached from an already-owned neighbor unless the disconnected-graph safety net fires. The remaining publication gap is empirical: NC/WI/TX quality and warm-start claims need archived same-metric run packages before they carry standalone weight. Implementation: `-structure bfs-growth`.

1. Introduction

1.1. The Smallest Useful Splitter

BISECT has several ways to split a bisection-tree node: METIS, GeoSection, CVD, simulated annealing, flow, exact methods, and others. BFSGROWTH is the opposite end of that spectrum. It asks what happens if a split is built with only the tract graph, tract populations, and a deterministic seed stream.

At each node, BFSGROWTH does four things:

1. choose one seed by population-weighted sampling;
2. choose the second seed at maximum BFS distance from the first;
3. grow two frontier regions, always expanding the side with smaller current population first; and
4. apply a local boundary rebalance pass.

This makes BFSGROWTH a useful baseline because it is easy to explain and easy to audit. It does not claim to optimize edge cut, Polsby-Popper, Reock, partisan fairness, or statutory compliance. It is a deliberately small contiguous growth rule that can be compared against stronger algorithms.

1.2. Show the Growth

The core idea should be visible, not merely named. In the schematic below, the two seeds start on opposite graph ends. The smaller side expands first. The frontier moves only through adjacency.

1. Seeds					2. Frontier					3. Split				
S0	.	.	.	S1	0	0	.	1	1	0	0	0	1	1
.	.	.	.	.	0	.	.	.	1	0	0	1	1	1
.	.	.	.	.	.	.	.	.	.	0	0	1	1	1

This is why BFSGROWTH belongs in the atlas and paper set: it gives a reader a plain visual model for what “region growing” means before the more complex optimization families enter the story.

1.3. Implemented Scope

The current Rust implementation is a recursive bisection structure: `run_all_splits_bfs` walks the standard `BisectionTree` and calls `split_subgraph_bfs` at each internal node. It does not grow k districts simultaneously. Odd-seat nodes inherit the same broad issue seen in CVD: the bisection tree records child seat counts, but this splitter’s local rebalance is still oriented around a half-population split.

The implementation uses two seed functions:

- `BFS_NODE_` derives the per-node seed from base seed and node path.
- `BFS_SEED_` is then used inside the split to seed the population-weighted first-seed sample.

That two-stage seed derivation should be preserved in any audit package: same base seed plus same node path implies same seed placement and same split.

1.4. What This Paper Claims

This paper makes three modest claims.

Algorithmic claim. `BFSGROWTH` is a deterministic, seed-auditable bisection primitive once the base seed and node path are fixed.

Reader claim. `BFSGROWTH` is the easiest construction algorithm to teach: seed, grow, rebalance. It should be used as the first visual baseline in explanations of the structure-layer family.

Evidence claim. The code has useful L0/L1 tests for validity, determinism, contiguity, and CLI parsing. Statewide quality and MCMC warm-start claims remain benchmark targets until archived run packages exist.

2. Background

2.1. BFS-Based Partitioning in the Multilevel Literature

Breadth-first search partitioning appears as an internal component of several multilevel graph partitioning frameworks. Karypis and Kumar [1998] describe BFS-based initial partitioning in Section 4 of the METIS paper: after the graph has been coarsened to a few hundred vertices, a BFS from two seeds produces an initial bisection that is immediately refined by Kernighan-Lin boundary swaps. The BFS step’s output quality is deliberately not optimised—it exists only to give KL a feasible starting point.

The use of BFS as a *standalone* partitioner—without subsequent KL refinement—has not been studied as a redistricting method. This paper fills that gap by evaluating BFS growth as a complete Structure-layer algorithm.

2.2. Two-Seed Farthest Selection

The quality of a BFS partition is determined primarily by seed placement. Seeds that are close together produce uneven districts (one large, one small); seeds at opposite ends of the graph produce more balanced initial boundaries.

The implemented two-seed strategy selects seeds greedily by BFS distance:

1. Seed 0 is selected by population-weighted sampling (denser areas first, reflecting the fact that high-population areas are more likely to require splitting and thus benefit from being starting points).
2. The second seed is the tract with maximum BFS distance from seed 0.

This is a graph-distance analogue of farthest-point initialisation [Cormen et al., 2009], but the implemented form is binary because BISECT grows plans through recursive bisection.

2.3. The Population-Deficit Priority Function

Once seeds are placed, the growth order determines the final district shapes. BFSGROWTH uses a *minimum-priority queue* keyed on current district population:

$$\text{priority}(\text{district } d) = P_d^{\text{current}},$$

Table 1: Comparison of structure-layer algorithms.

Property	BFSGROWTH	CVD	METIS
Metric optimised	None (greedy)	Geographic distance	Edge cut
Data needed	Adjacency only	Adjacency + centroids	Adjacency
Iterations	1 pass	~ 20 iters	Multilevel KL
Runtime	$O(n \log n)$	$O(n \times k \times 20)$	$O(n \log n)$
Contiguity guarantee	Yes (BFS)	No (post-process)	No (post-process)
Compactness	Benchmark target	Proximity packing	Edge-cut oriented
Hyperparameters	0	1 (iter count)	1 (ufactor)

where P_d^{current} is the total population already assigned to district d . The tract with the lowest priority key is expanded next; ties are broken by tract index.

This priority function fills the *currently smaller* side first. It is motivated by the following balance argument: if district d has population far below the ideal, its neighbours should be assigned to d before they are assigned to a more populated district. The priority is a heuristic, not a balance proof. Large tract populations, frontier geometry, disconnected components, and the later rebalance pass all affect the final deviation.

Comparison with CVD. CVD assigns each tract to the district whose centroid (geographic center of mass) is closest, iterating until centroid positions stabilise [Du et al., 1999]. The centroidal Voronoi equilibrium minimises within-district sum of squared geographic distances—a proxy for compactness.

BFSGROWTH assigns tracts by BFS adjacency order, with population deficit breaking ties. It does not compute centroids and does not iterate. The single-pass nature means that BFSGROWTH cannot recover from early suboptimal assignments, but it also means that BFSGROWTH runs in $O(n \log n)$ rather than $O(n \times k \times \text{iter})$.

Table 1 summarises the relationship between BFSGROWTH, CVD, and METIS along the key dimensions.

2.4. Structure-Layer Context in the B-Series Platform

The B-series redistricting platform uses a three-layer compositor: *Structure* (how each bisection node is solved), *WeightsOverride* (what edge weights encode), and *SeedCompositor* (how many seeds are tried). BFSGROWTH is a Structure-layer algorithm.

Because BFSGROWTH is a Structure-layer algorithm, it composes cleanly with all SeedCompositor strategies. Running BISECTIONENSEMBLE with $T = 50$ seeds, each using BFSGROWTH growth, selects the best of 50 BFS growth outcomes. Running CONVERGENCESWEEP with BFSGROWTH sweeps forward through seeds until $T = 600$ consecutive seeds produce no improvement. In both cases, the SeedCompositor provides the multi-seed search quality improvement that BFSGROWTH alone cannot.

Existing structure-layer algorithms all modify *what is optimised* at each node: GEosection [Della-Libera, 2026b] minimises normalised edge cut; AREasection adds area-swing constraints; APPORTIONREGIONS [Della-Libera, 2026a] links the bisection tree to Huntington-Hill apportionment. BFSGROWTH is the structure-layer algorithm that optimises the least: it is the minimal adjacency-growth baseline against which stronger splitters can be measured once same-metric packages exist.

3. The BFS Region-Growing Algorithm

3.1. Notation and Subgraph Setup

Let $G_v = (V_v, E_v)$ be the tract subgraph at bisection-tree node v . Let p_i be tract population and $P_v = \sum_{i \in V_v} p_i$ be node population. The implemented split is binary: it returns a left set L and a right set R whose union is V_v .

3.2. Seed Derivation

Definition 1 (BFS Node Seed). *For global `base_seed` and node path `path`, BISECT derives*

$$s_{\text{node}} = \text{first8}(\text{SHA-256}(\text{"BFS_NODE_"} \parallel \text{path} \parallel \text{"_"} \parallel \text{le8}(\text{base_seed}))).$$

Definition 2 (BFS Split Seed). *Inside `split_subgraph_bfs`, the node seed is domain-separated again:*

$$s_{\text{split}} = \text{first8}(\text{SHA-256}(\text{"BFS_SEED_"} \parallel \text{le8}(s_{\text{node}}))).$$

This seed initializes `SmallRng` for the population-weighted sample of the first seed.

3.3. Seed Selection

The split uses exactly two seeds.

1. **Seed 0.** Sample a tract from V_v with probability proportional to tract population, using the split seed from Definition 2.
2. **Seed 1.** Run BFS from seed 0 and choose a tract with maximum finite BFS distance. Ties follow the deterministic iterator order over local tract indices.

The result is not a geometric centroid. It is a graph-topological spread: one seed starts in a population-weighted place, and the other is as far away as the subgraph permits in hop distance.

3.4. Priority-Queue Growth

Each seed owns itself initially. The algorithm pushes unassigned neighbors of each seed into a binary heap keyed by current district population. Because Rust’s `BinaryHeap` is a max-heap, the implementation stores `Reverse(current_pop)` so the smaller-population side is popped first.

Definition 3 (BFS Growth Step). *When a heap entry $(\text{current_pop}, t, d)$ is popped:*

1. *if tract t is already assigned, discard the stale entry;*
2. *otherwise assign t to side d ;*
3. *add p_t to side d ’s current population; and*
4. *push every still-unassigned neighbor of t back into the heap with side d ’s updated population.*

The stale-entry rule matters. A tract can be reached from both frontiers; the first pop wins, later heap entries for the same tract are ignored.

Algorithm 1 BFSGROWTH-BISECT($G_v, \mathbf{p}, \text{base_seed}, \text{path}$)

```
1:  $s_{\text{node}} \leftarrow \text{BFSNodeSeed}(\text{base\_seed}, \text{path})$ 
2:  $s_{\text{split}} \leftarrow \text{BFSSplitSeed}(s_{\text{node}})$ 
3:  $\text{seed}_0 \leftarrow \text{WeightedSample}(V_v, \mathbf{p}, s_{\text{split}})$ 
4:  $\text{seed}_1 \leftarrow \arg \max_t d_{\text{BFS}}(\text{seed}_0, t)$ 
5: assign seed 0 to side 0 and seed 1 to side 1
6: push their unassigned neighbors into a heap keyed by side population
7: while heap is non-empty do
8:   pop the entry whose side has smaller current population
9:   if tract is already assigned then continue
10:  end if
11:  assign tract to that side and update side population
12:  push unassigned neighbors for the same side
13: end while
14: assign disconnected leftovers to the currently smaller side
15: apply up to 200 boundary rebalance moves toward  $P_v/2$ 
16: return left and right tract sets
```

3.5. Disconnected-Graph Safety Net

If the induced subgraph is disconnected, the frontier heap can empty before every tract is assigned. The implementation then assigns any leftover tract to the side with smaller current population. This makes the output complete, but it weakens any pure contiguity guarantee. Therefore the correct claim is: BFSGROWTH is contiguous by construction on connected subgraphs before the disconnected-graph safety net and subject to any later rebalance moves.

3.6. Post-Hoc Rebalance

After growth, the splitter performs up to 200 local boundary moves. At each iteration it computes left population, compares it to $P_v/2$, identifies the heavy side, and moves one boundary tract whose population best reduces the excess if such a boundary tract exists. The current implementation does not run a full contiguity check inside this rebalance loop; it uses boundary adjacency as the local repair signal.

This is enough for a fast construction baseline, but not enough for a strong legal-style contiguity theorem. The final plan should still pass the platform's ordinary validation surface.

3.7. Formal Pseudocode

3.8. Runtime

On a connected node, each edge can generate heap activity from frontier growth, so the natural bound is $O((|V_v| + |E_v|) \log |V_v|)$ plus the bounded 200-step rebalance pass. On planar tract graphs this behaves like $O(n \log n)$ per bisection node. End-to-end statewide runtime depends on the bisection tree and validation pipeline.

4. Benchmark Targets

4.1. Evidence Status

The older T.12 draft contained specific NC/WI/TX compactness, edge-cut, runtime, and partisan tables. Those values are useful hypotheses, but this paper should not present them as final evidence until the corresponding run directories, source hashes, seed manifests, and analysis outputs are archived.

The current verified evidence is narrower:

- small-grid BFS Growth returns a complete two-way split;
- repeated runs with the same seed are deterministic;
- assigned sides are contiguous on connected toy graphs before validation;
- the `bfs-growth` structure mode parses through the CLI surface; and
- `run_all_splits_bfs` produces complete assignments for toy $k=2$ cases.

4.2. Publication Matrix

4.3. What to Measure

The comparison should separate three questions.

Construction quality. How much compactness and edge-cut quality does the simplest adjacency-growth rule provide before heavier optimization is added?

Table 2: BFS Growth benchmark matrix. Values remain pending until archived same-metric packages are available.

State	Algorithm	PP	EC	Pop dev.	Package
NC ($k = 14$)	BFSGROWTH	TBD	TBD	TBD	pending
NC ($k = 14$)	METIS	TBD	TBD	TBD	pending
NC ($k = 14$)	CVD	TBD	TBD	TBD	pending
WI ($k = 8$)	BFSGROWTH	TBD	TBD	TBD	pending
WI ($k = 8$)	METIS	TBD	TBD	TBD	pending
WI ($k = 8$)	CVD	TBD	TBD	TBD	pending
TX ($k = 38$)	BFSGROWTH	TBD	TBD	TBD	pending
TX ($k = 38$)	METIS	TBD	TBD	TBD	pending
TX ($k = 38$)	CVD	TBD	TBD	TBD	pending

Validation cost. How often does boundary rebalance leave a plan that final validation must repair or reject, especially on disconnected or island-heavy graphs?

Odd-seat behavior. For odd-seat bisection nodes, how different is a half-population BFS split from the bisection tree’s intended $k_L : k_R$ child ratio?

4.4. Reader-Facing Example

The benchmark package should include at least one worked tract-block example showing every heap pop:

Step	Pop 0	Pop 1	Assigned tract
0	10	12	seeds only
1	18	12	side 0 grows
2	18	20	side 1 grows
3	25	20	side 0 grows
4	25	27	side 1 grows

This is the visual standard for T.12: show the heap decision, not just the final map.

5. Warm-Start Role

5.1. Why It Might Help

MCMC and ReCom-style workflows need an initial plan. A METIS warm-start is standard because it is fast and tends to have low edge cut. A BFSGROWTH warm-start is different: it is also fast, but its boundaries are produced by frontier growth rather than multilevel cut refinement. That can be useful when the goal is to test sensitivity to starting geography.

5.2. What Is Not Yet Proven

The paper should not yet claim that BFSGROWTH improves burn-in, effective sample size, or autocorrelation decay. Those claims require paired chains from METIS and BFS Growth starts, identical proposal kernels, identical seeds where applicable, archived traces, and diagnostic summaries.

5.3. Warm-Start Test Plan

A proper warm-start package should record:

- initial plan EC, PP, population deviation, and district contiguity;
- chain proposal type and all proposal seeds;
- accepted/rejected proposal ledger;
- EC and compactness traces;
- effective sample size and R-hat when multiple chains are run; and
- final comparison between METIS-started and BFS-started chains.

Until that package exists, the practical recommendation is conservative: METIS remains the default warm-start, and BFSGROWTH is an alternate starting condition for robustness checks.

5.4. CLI Shape

```
bisect state --state NC --year 2020 \  
--structure bfs-growth --search single
```

The resulting plan can be used wherever the platform accepts a previously generated plan as an initial condition. The important audit fields are the base seed, node path, BFS node seed, BFS split seed, adjacency hash, and population hash.

6. Conclusion

BFSGROWTH is valuable because it is small. It gives the BISECT family a minimal, seed-auditable, adjacency-only construction baseline: choose two spread seeds, grow the smaller side first, fill disconnected leftovers, and rebalance locally.

Correct role. BFSGROWTH should be described as a baseline and warm-start candidate, not as a legal certificate and not as a proven compactness floor. Strong compactness, runtime, and partisan claims belong in archived benchmark packages.

Implementation caveats. The current implementation is recursive two-way bisection, not simultaneous k -way growth. The disconnected-graph safety net and local rebalance step mean the final plan still needs ordinary validation. Odd-seat nodes also need future ratio-aware target handling if BFSGROWTH is to make strong claims for bisection trees with uneven child seat counts.

Next work. The next useful milestones are:

1. archive NC/WI/TX same-metric packages for BFSGROWTH, METIS, and CVD;
2. add a worked heap-pop trace to the public algorithm atlas;
3. decide whether ratio-aware split targets should be shared across CVD, BFS Growth, and other recursive splitters; and
4. run paired warm-start chain experiments before making any MCMC convergence claims.

In the algorithm family, BFSGROWTH is the first thing to teach after a plain graph: it shows how a district can grow from a seed by adjacency alone. That clarity is its main contribution.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009. Standard reference for BFS ($O(V + E)$ time) and binary heap priority queues ($O(\log n)$ per operation). The $O(n \log n)$ complexity of BFS Growth follows from n priority-queue operations during the growth phase.
- Gio Della-Libera. ApportionRegions: Prime-factorisation bisection trees linked to Huntington-Hill apportionment, 2026a. T.4 in Algorithmic Redistricting series. Establishes the prime-factor tree structure used as an alternative compositor configuration. BFS Growth composes with the ApportionRegions tree; this is identified as future work in T.12.
- Gio Della-Libera. GeoSection: Isoperimetrically-normalised ratio-optimal bisection for congressional redistricting, 2026b. T.1 in Algorithmic Redistricting series. Introduces the GEOSECTION structure-layer algorithm. Empirical PP baselines (NC 0.217, WI 0.198, TX 0.184) used as the METIS-only baseline in T.12 Table 1.
- Qiang Du, Vance Faber, and Max Gunzburger. Centroidal Voronoi tessellations: Applications and algorithms. *SIAM Review*, 41(4):637–676, 1999. doi: 10.1137/S0036144599352836. Foundational CVD paper. Iterative Lloyd’s algorithm alternates Voronoi assignment (geographic proximity) and centroid update. BFS Growth is the non-iterative analogue: a single BFS pass with population-deficit priority replaces Lloyd’s iterations. CVD converges to a lower-distortion solution at higher computational cost.
- George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998. doi: 10.1137/S1064827595287997. Original METIS paper. Section 4 describes BFS-based initial partitioning as an internal warm-start for the coarsening phase. T.12 elevates this internal mechanism to a standalone algorithm. Establishes $O(m \log n)$ complexity of multilevel KL partitioning.